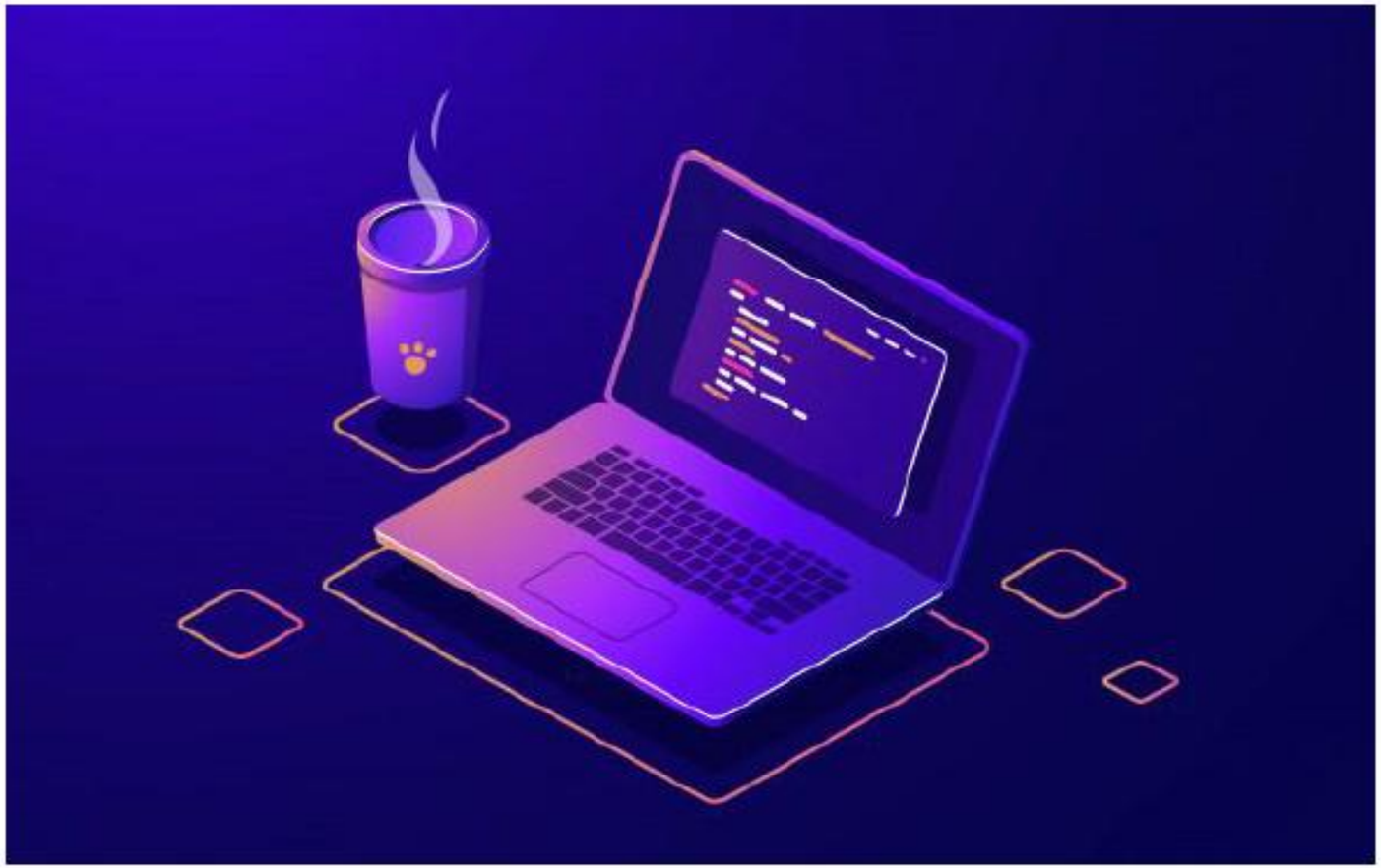


# PROGRAMACIÓN

# TEMA 6: LECTURA Y ESCRITURA DE INFORMACIÓN

# LECTURA Y ESCRITURA DE INFORMACIÓN

## INDICE


1. Persistencia de datos en aplicaciones
2. Sistemas de archivos y rutas (Path)
3. API moderna java.nio.file
4. Flujos de datos: concepto y tipos
5. API clásica java.io (uso controlado)
6. Acceso a datos binarios
7. Acceso directo a archivos (RandomAccessFile)
8. Persistencia de objetos (serialización)
9. Persistencia moderna con JSON

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 1. PERSISTENCIA DE DATOS

### 1.1 ¿Qué significa persistir información?

**Persistir información** consiste en **almacenar datos de forma permanente**, de manera que:

- No se pierdan al cerrar la aplicación.
- No dependan de la memoria RAM.
- Puedan recuperarse en ejecuciones posteriores.

En términos prácticos:

Un dato persistente es aquel que **existe más allá de la ejecución del programa**.

Esto marca una diferencia fundamental entre:

- Programas de ejemplo o ejercicios simples.
- Aplicaciones reales de uso profesional.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 1. PERSISTENCIA DE DATOS

### 1.2 Persistencia y ciclo de vida de los datos

Durante la ejecución de un programa, los datos pasan por distintas fases:

1. **Creación** (entrada por teclado, formulario, API, etc.)
2. **Uso en memoria** (variables, objetos)
3. **Persistencia** (almacenamiento externo)
4. **Recuperación** (lectura posterior)

Si un dato no se persiste:

- Se pierde al finalizar el programa.
- La aplicación no puede considerarse completa ni realista.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 1. PERSISTENCIA DE DATOS

### 1.3 Ejemplos reales

**1. Guardar usuarios registrados**

Una aplicación web o de escritorio necesita:

- Conservar usuarios.
- Recordar credenciales, roles o preferencias.
- Recuperar esa información en futuras sesiones.

Sin persistencia:

- El registro de usuarios carece de sentido.

**2. Almacenar configuración de una aplicación**

Ejemplos habituales:

- Idioma seleccionado.
- Tema visual.
- Rutas de trabajo.
- Parámetros de conexión.

Estos datos:

- No cambian constantemente.
- Deben sobrevivir a reinicios del programa.
- Suelen persistirse en **ficheros de configuración** (JSON, properties, XML).

**3. Exportar datos para otros sistemas**

Casos frecuentes en DAW:

- Exportar listados.
- Generar informes.
- Intercambiar información con otras aplicaciones.

Aquí la persistencia:

- No es solo para la propia aplicación.
- Sirve como **mecanismo de comunicación entre sistemas**.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 1. PERSISTENCIA DE DATOS

### 1.4 Tipos de persistencia según su duración

**Persistencia temporal**

Datos que:

- Solo existen mientras el programa está en ejecución.
- Se almacenan en **memoria RAM**.

Ejemplos:

- Variables locales.
- Atributos de objetos.
- Colecciones en memoria.

Características:

- Muy rápida.
- Muy volátil.
- Se pierde al cerrar la aplicación.

**No es persistencia real**, aunque sea imprescindible para el funcionamiento interno.

**Persistencia permanente**

Datos que:

- Se almacenan fuera de la memoria RAM.
- Pueden recuperarse posteriormente.

Es el tipo de persistencia que interesa en DAW.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 1. PERSISTENCIA DE DATOS

### 1.5 Formas de persistencia permanente

**1. Ficheros**

Almacenamiento directo en el sistema de archivos.

Tipos:

- **Texto** (JSON, CSV, TXT, XML)
- **Binario** (datos compactos, serialización, formatos propios)

Ventajas:

- Simples de usar.
- No requieren infraestructura adicional.
- Ideales para:
  - Configuración.
  - Exportación/importación.
  - Persistencia ligera.

Limitaciones:

- No gestionan bien grandes volúmenes.
- No controlan concurrencia.
- No ofrecen consultas complejas.

**2. Bases de datos**

Almacenamiento estructurado y persistente.

Características:

- Datos relacionales o no relacionales.
- Consultas eficientes.
- Control de integridad y concurrencia.

En DAW:

- Son la **solución principal** para datos de negocio.
- Se estudian en bloques posteriores (JDBC, ORM).

**3. Servicios externos (APIs, cloud)**

Persistencia delegada a servicios externos:

- APIs REST.
- Servicios en la nube.
- Sistemas distribuidos.

Características:

- Persistencia remota.
- Escalabilidad.

- Comunicación mediante formatos estándar (JSON).

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 1. PERSISTENCIA DE DATOS

### 1.5 Formas de persistencia permanente

**Los ficheros no son la solución universal.**

- Los ficheros **no sustituyen** a una base de datos.
- No todo debe guardarse en archivos.
- El programador debe **elegir la técnica adecuada** según el problema.

Ejemplo de criterio correcto:

- Configuración → fichero
- Exportación → fichero
- Datos de usuarios → base de datos
- Comunicación entre sistemas → JSON / API

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 1. PERSISTENCIA DE DATOS

### 1.5 Formas de persistencia permanente

**Almacenamiento en ficheros**

Los ficheros permiten guardar información directamente en el sistema de archivos. Pueden contener datos en formato texto o binario.

En los primeros enfoques de programación, la información se organizaba en **registros**, entendidos como conjuntos de datos relacionados, formados por distintos campos.

Ejemplo de registro:

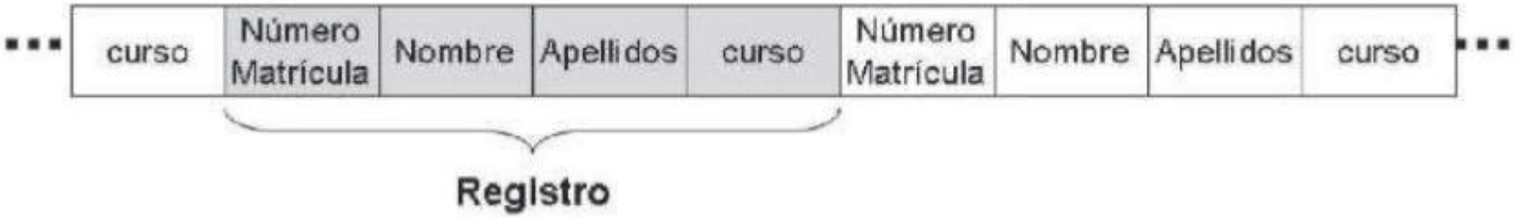
- Nombre
- Edad
- Correo electrónico

**Idea clave**

La persistencia de datos implica un proceso de transformación:

**Registro → Objeto → Formato persistente**

Comprender esta evolución permite entender cómo se almacenan y recuperan los datos en una aplicación, y prepara el terreno para el estudio de los mecanismos concretos de lectura y escritura de información en ficheros.



# LECTURA Y ESCRITURA DE INFORMACIÓN

## 1. PERSISTENCIA DE DATOS

### 1.5 Formas de persistencia permanente

**Del registro al objeto**

En programación orientada a objetos, el concepto de registro se representa mediante **objetos**:

- El registro se convierte en una **clase**.
- Los campos del registro pasan a ser **atributos** del objeto.
- Cada instancia de la clase representa un conjunto concreto de datos.

De este modo, la información deja de tratarse como estructuras planas y pasa a gestionarse mediante objetos.

**Del objeto al formato persistente**

Para almacenar un objeto de forma permanente, es necesario **transformarlo en un formato que pueda guardarse en un fichero**.

Existen distintas posibilidades:

- Representarlo como datos binarios.
- Convertirlo a un formato de texto estructurado.

Una de las formas más habituales de representar objetos en ficheros de texto es **JSON**, que permite expresar la información de un objeto mediante pares clave–valor.

Ejemplo conceptual:

- Registro → conjunto de campos
- Objeto → conjunto de atributos
- JSON → representación textual del objeto

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 2. SISTEMAS DE ARCHIVOS Y RUTAS.

### 2.1 El Sistema de Archivos.

Un **sistema de archivos** es el mecanismo que utiliza un sistema operativo para **organizar, almacenar y localizar información** de forma permanente en un dispositivo de almacenamiento.

Su función principal es:

- Almacenar datos de forma estructurada.
- Permitir su acceso posterior.
- Gestionar la relación entre archivos y directorios.

Desde el punto de vista del programador, el sistema de archivos es el **entorno donde se persisten los datos** de una aplicación.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 2. SISTEMAS DE ARCHIVOS Y RUTAS.

### 2.2 Archivos y Directorios.

**Archivos**

Un **archivo** es un contenedor de información almacenada de forma permanente. Puede contener:

- Texto (caracteres legibles).
- Datos binarios (bytes sin interpretación directa).

Cada archivo se identifica mediante:

- Un nombre.
- Una ruta que indica su ubicación.
- Un formato o extensión, que suele indicar el tipo de contenido.

**Directorios**

Un **directorio** es un contenedor que permite **organizar archivos y otros directorios**.

Los directorios:

- No almacenan datos directamente (en el sentido lógico).
- Facilitan la clasificación y localización de la información.
- Permiten estructurar los datos de una aplicación.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 2. SISTEMAS DE ARCHIVOS Y RUTAS.

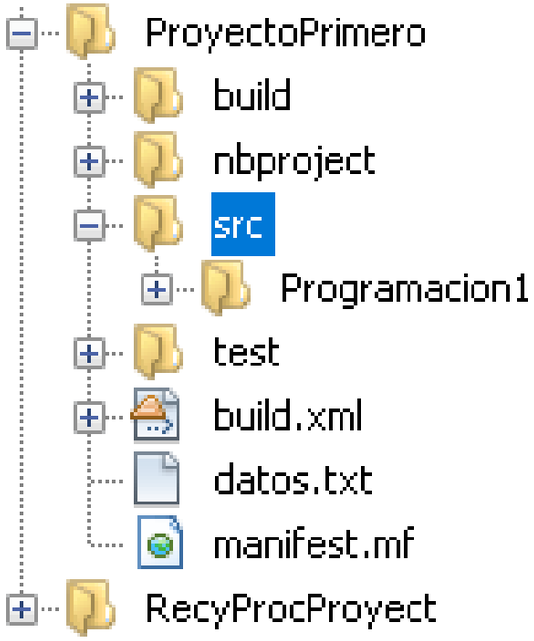
### 2.3 Organización Jerárquica del Sistema de Archivos.

El sistema de archivos tiene una **estructura jerárquica**, similar a un árbol:

- Existe un directorio raíz.
- Cada directorio puede contener archivos y subdirectorios.
- La jerarquía define la ubicación exacta de cada archivo.

Esta estructura permite:

- Evitar conflictos de nombres.
- Organizar grandes volúmenes de información.
- Acceder a los datos de forma precisa.



# LECTURA Y ESCRITURA DE INFORMACIÓN

## 2. SISTEMAS DE ARCHIVOS Y RUTAS.

### 2.4 Rutas: Localización de Archivos y Directorios.

Una **ruta** es la representación textual de la ubicación de un archivo o directorio dentro del sistema de archivos.

Las rutas permiten a un programa:

- Localizar un recurso concreto.
- Acceder a él para leer o escribir información.

**Rutas absolutas**

Una **ruta absoluta** indica la ubicación completa de un archivo desde el directorio raíz del sistema.

Características:

- Es independiente del punto desde el que se ejecute el programa.
- Identifica de forma única un archivo.

Ejemplo conceptual:

- `/home/usuario/datos/config.json`
- `C:\\Users\\Usuario\\datos\\config.json`

**Rutas relativas**

Una **ruta relativa** indica la ubicación de un archivo **en relación con el directorio de trabajo** del programa.

Características:

- Depende del contexto de ejecución.
- Es más flexible y portable.
- Recomendable para proyectos y aplicaciones.

Ejemplo conceptual: `datos/config.json`

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 2. SISTEMAS DE ARCHIVOS Y RUTAS.

### 2.5 Problemas de Portabilidad.

Los sistemas operativos utilizan:

- Distintos separadores de directorios.
- Diferentes convenciones de rutas.

Esto puede provocar problemas si las rutas se construyen manualmente como cadenas de texto.

Ejemplo de problema común:

- Uso directo de / o \\ en el código.
- Dependencia del sistema operativo.

### 2.6 Abstracción de rutas

Para evitar estos problemas, los lenguajes modernos proporcionan **abstracciones de ruta** que:

- Representan rutas de forma independiente del sistema operativo.
- Gestionan correctamente separadores y convenciones internas.

Estas abstracciones permiten:

- Construir rutas de forma segura.
- Aumentar la portabilidad del código.
- Reducir errores.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 2. SISTEMAS DE ARCHIVOS Y RUTAS.

### 2.7 Directorio de trabajo de una aplicación

Toda aplicación se ejecuta desde un **directorio de trabajo**, que sirve como referencia para:

- Resolver rutas relativas.
- Ubicar archivos de entrada y salida.

Comprender el directorio de trabajo es fundamental para:

- Evitar errores de localización.
- Organizar correctamente los recursos de una aplicación.

### 2.8 Importancia de las rutas en la persistencia

La persistencia de datos depende directamente de:

- Dónde se almacenan los archivos.
- Cómo se accede a ellos mediante rutas.

Una mala gestión de rutas puede provocar:

- Pérdida de datos.
- Dificultad para localizar información.
- Problemas al ejecutar la aplicación en distintos entornos.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 3. REPRESENTACIÓN DE DATOS EN FICHEROS: TEXTO Y BINARIO.

### 3.1 La necesidad de representar los datos

Para que la información pueda almacenarse en un fichero, debe transformarse en una **secuencia de datos compatible con el sistema de archivos**.

Independientemente de su origen (variables, objetos, estructuras), toda información persistente termina representándose como:

- **Caracteres** (texto), o
- **Bytes** (binario).

La forma elegida condiciona:

- Cómo se escribe la información.
- Cómo se lee posteriormente.
- Qué herramientas pueden utilizarse para interpretarla.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 3. REPRESENTACIÓN DE DATOS EN FICHEROS: TEXTO Y BINARIO.

### 3.2 Representación en formato texto

Un fichero de texto almacena la información como una **secuencia de caracteres**, siguiendo una codificación determinada.

**Características principales**

- Contenido legible por personas.
- Puede abrirse con editores de texto.
- Utiliza codificaciones de caracteres (UTF-8, UTF-16, etc.).

Ejemplos habituales de ficheros de texto:

- `.txt`
- `.csv`
- `.json`
- `.xml`

**Ventajas del formato texto**

- Facilidad de lectura y depuración.
- Portabilidad entre sistemas.
- Interoperabilidad entre aplicaciones.
- Adecuado para intercambio de información.

**Limitaciones del formato texto**

- Mayor tamaño que el binario.
- Procesamiento más costoso.
- Menor eficiencia en grandes volúmenes de datos.

### 3.3 Codificación de caracteres

La codificación define **cómo se representa cada carácter como uno o varios bytes**.

Aspectos clave:

- Un mismo texto puede ocupar distinto espacio según la codificación.
- Una codificación incorrecta provoca errores de lectura.

La codificación más utilizada actualmente es:

- **UTF-8**, por su compatibilidad y eficiencia.

Comprender la codificación es esencial para evitar:

- Caracteres incorrectos.
- Pérdida de información.
- Problemas de portabilidad.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 3. REPRESENTACIÓN DE DATOS EN FICHEROS: TEXTO Y BINARIO.

### 3.4 Representación en formato binario

Un fichero binario almacena información como **secuencias de bytes**, sin interpretación directa como caracteres.

En este formato:

- Los datos se escriben tal como están en memoria.
- La interpretación depende del programa que los lee.

Ejemplos de datos binarios:

- Números en formato binario.
- Estructuras compactas.
- Archivos multimedia.
- Formatos propietarios.

**Ventajas del formato binario**

- Menor tamaño.
- Mayor eficiencia.
- Lectura y escritura más rápidas.

**Limitaciones del formato binario**

- No es legible por personas.
- Depende del formato exacto.
- Requiere conocer el orden y tipo de los datos.
- Menor interoperabilidad.

### 3.6 Relación con el concepto de registro y objeto

Un mismo conjunto de datos puede representarse de distintas formas:

- Como **texto estructurado**, donde cada campo se expresa como caracteres.
- Como **binario**, donde cada campo ocupa una posición concreta.

En ambos casos:

- El conjunto de datos representa una entidad lógica.
- La diferencia está en **cómo se codifica y se almacena**.

Esta decisión afecta directamente al modo de lectura y escritura del fichero.

### 3.7 Criterio de elección del formato

El formato debe elegirse en función de:

- Quién va a leer el fichero.
- Cuántos datos se almacenan.
- Con qué frecuencia se accede a ellos.
- Si es necesario intercambiar información con otros sistemas.

Ejemplos de criterio:

- Intercambio de datos → texto.
- Configuración → texto.
- Datos compactos y controlados → binario.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 4. API MODERNA DE ACCESO A ARCHIVOS (*java.nio.file*).

### 4.1 Introducción a la API moderna

Java proporciona, desde la versión 7, una API moderna para trabajar con el sistema de archivos: el paquete **`java.nio.file`**.

Esta API permite:

- Acceder a archivos y directorios.
- Leer y escribir información.
- Gestionar rutas de forma portable.
- Reducir la complejidad del código.

A diferencia de enfoques más antiguos, esta API ofrece **operaciones de alto nivel**, adecuadas para la mayoría de los casos habituales.

### 4.2 Clases principales de `java.nio.file`

Las clases fundamentales son:

- **`Path`** Representa la ruta a un archivo o directorio.
- **`Paths`** Clase de utilidad para crear objetos `Path`.
- **`Files`** Contiene métodos estáticos para operar sobre archivos.

Estas clases abstraen los detalles del sistema operativo y facilitan el acceso a los datos.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 4. API MODERNA DE ACCESO A ARCHIVOS (*java.nio.file*).

### 4.3 Creación y uso de rutas (`Path`)

Un objeto `Path` representa **la ubicación de un recurso en el sistema de archivos**.

Ejemplo:

```
Path ruta = Paths.get("datos", "config.txt");
```

Características:

- No accede al archivo todavía.
- Solo representa la ruta.
- Es independiente del sistema operativo.

El uso de `Path` evita construir rutas manualmente como cadenas de texto.

### 4.4 Lectura de archivos de texto completos

Para archivos de texto de tamaño moderado, la API moderna permite leer el contenido completo de forma sencilla.

**Leer todo el contenido como una cadena**

```
Path ruta = Paths.get("datos.txt");
String contenido = Files.readString(ruta);
System.out.println(contenido);
```

Qué hace este código:

- Abre el archivo indicado por la ruta.
- Lee todo su contenido.
- Devuelve una cadena (`String`).

Este enfoque es adecuado cuando:

- El archivo no es muy grande.
- Se necesita procesar el contenido completo.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 4. API MODERNA DE ACCESO A ARCHIVOS (*java.nio.file*).

### 4.4 Lectura de archivos de texto completos

**Leer todas las líneas en una lista**

```
Path ruta = Paths.get("datos.txt");
List<String> lineas =
Files.readAllLines(ruta);

for (String linea : lineas) {
    System.out.println(linea);
}
```

En este caso:

- Cada línea del archivo se almacena como un elemento de la lista.
- Facilita el procesamiento línea a línea posterior.

### 4.5 Escritura de Archivos de Texto.

La API moderna también permite escribir texto de forma directa.

**Escribir un texto completo**

```
Path ruta = Paths.get("salida.txt");
Files.writeString(ruta, "Texto de ejemplo");
```

Este método:

- Crea el archivo si no existe.
- Sobrescribe su contenido si ya existe.

**Escribir varias líneas**

```
List<String> lineas = List.of(
    "Primera línea",
    "Segunda línea",
    "Tercera línea"
);
Path ruta = Paths.get("salida.txt");
Files.write(ruta, lineas);
```

Este enfoque es útil para:

- Generar informes.
- Exportar datos estructurados simples.
- Guardar información legible.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 4. API MODERNA DE ACCESO A ARCHIVOS (*java.nio.file*).

### 4.5.1 Escritura en modo append (sin sobrescribir)

- Comportamiento por defecto: sobrescribe
- Para añadir contenido → opciones de apertura

```
Files.writeString(
    ruta,
    "Nueva línea\n",
    StandardOpenOption.CREATE,
    StandardOpenOption.APPEND
);
```

**Sin `APPEND`, Java sobrescribe siempre el archivo.**

### 4.6 Gestión de excepciones

Las operaciones de lectura y escritura pueden fallar por distintos motivos:

- Archivo inexistente.
- Permisos insuficientes.
- Errores del sistema.

Por ello, los métodos de `Files` lanzan **`IOException`**, que debe gestionarse.

Ejemplo básico:

```
try {
    String texto = Files.readString(ruta);
    System.out.println(texto);
} catch (IOException e) {
    System.out.println("Error al leer el
archivo");
}
```

La gestión de excepciones es parte esencial del acceso a archivos.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 4. API MODERNA DE ACCESO A ARCHIVOS (*java.nio.file*).

```java
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.List;

public class NioFilesExample {
    public static void main(String[] args) {
        Path filePath = Paths.get("archivo.txt");

        // Escribir en el archivo usando java.nio.file
        escribirEnArchivo(filePath);

        // Leer desde el archivo usando java.nio.file
        leerDesdeArchivo(filePath);
    }

    public static void escribirEnArchivo(Path filePath) {
        List<String> lineas = List.of(
            "Hola, este es un archivo creado con java.nio.file.",
            "Segunda línea del archivo.",
            "Tercera línea con más contenido."
        );

        try {
            Files.write(filePath, lineas, StandardCharsets.UTF_8);
            System.out.println("Datos escritos correctamente en el archivo.");
        } catch (IOException e) {
            System.err.println("Error al escribir en el archivo: " + e.getMessage());
        }
    }
```

```java
    public static void leerDesdeArchivo(Path filePath) {
        try {
            List<String> lineas = Files.readAllLines(filePath,
                                                     StandardCharsets.UTF_8);
            System.out.println("\nContenido del archivo:");
            for (String linea : lineas) {
                System.out.println(linea);
            }
        } catch (IOException e) {
            System.err.println("Error al leer el archivo: "
                                                    + e.getMessage());
        }
    }
}
```

Método **escribirEnArchivo(Path filePath)**
Define una lista de cadenas con el contenido a escribir (List.of(...)).
Usa Files.write() para escribir el contenido en el archivo de manera sencilla y eficiente.
Utiliza StandardCharsets.UTF_8 para asegurarse de que el texto se guarda correctamente.

Método **leerDesdeArchivo(Path filePath)**
Usa Files.readAllLines() para leer todas las líneas del archivo en una lista de String.
Itera sobre la lista e imprime el contenido en la consola.

Método **main()**
Primero escribe datos en el archivo llamando a escribirEnArchivo().
Luego lee los datos del archivo llamando a leerDesdeArchivo().

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 4. API MODERNA DE ACCESO A ARCHIVOS (*java.nio.file*).

### 4.7 Relación con la representación de datos

Cuando se utiliza `java.nio.file` de esta forma:

- Se trabaja **principalmente con texto**.
- Los datos se manejan como cadenas o listas de cadenas.
- La codificación de caracteres se gestiona internamente (por defecto, UTF-8).

Este enfoque encaja especialmente bien con:

- Datos legibles.
- Formatos estructurados de texto.
- Persistencia sencilla.

### 4.8 Limitaciones de la API moderna

Aunque muy potente, esta API no cubre todos los escenarios:

- No es adecuada para archivos muy grandes.
- No permite control fino del proceso de lectura.
- No gestiona directamente datos binarios complejos.

Estas limitaciones justifican la existencia de otros mecanismos, que se estudiarán más adelante.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 5. CONCEPTO DE FLUJOS DE DATOS.

### 5.1 Qué es un flujo de datos

Un **flujo de datos** representa el **camino por el que viaja la información** entre un origen y un destino.

Desde el punto de vista de un programa:

- El origen puede ser un archivo, un teclado, una red o la memoria.
- El destino puede ser un archivo, la pantalla, una red o la memoria.

El flujo abstrae los detalles del origen y del destino, permitiendo tratar la entrada y la salida de datos de forma uniforme.

### 5.2 Flujos de entrada y flujos de salida

Según la dirección de los datos, existen dos tipos de flujos:

- **Flujos de entrada**: permiten leer datos hacia el programa.
- **Flujos de salida**: permiten escribir datos desde el programa.

El programa no accede directamente al archivo, sino a través del flujo que lo conecta con él.

### 5.3 Flujos y acceso a archivos

Cuando se utiliza una API de alto nivel, como `java.nio.file`, los flujos:

- No aparecen explícitamente en el código.
- Se crean y gestionan internamente.

Por ejemplo, al leer un archivo completo:

- Se abre un flujo de entrada.
- Se leen los datos.
- El flujo se cierra automáticamente.

Comprender este mecanismo ayuda a entender **qué ocurre internamente** durante la lectura y escritura de archivos.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 5. CONCEPTO DE FLUJOS DE DATOS.

### 5.4 Clasificación de los flujos

Los flujos se clasifican según el tipo de datos que transportan.

**Flujos de bytes**

- Transportan datos como bytes (`0` y `1`).
- Son adecuados para:
  - Datos binarios.
  - Archivos no textuales.
- Trabajan con tipos básicos.

**Flujos de caracteres**

- Transportan datos como caracteres.
- Se apoyan en una codificación.
- Son adecuados para:
  - Texto legible.
  - Lectura por líneas.

### 5.5 Relación entre bytes y caracteres

Aunque se trabaje con caracteres:

- Internamente, toda la información se almacena como bytes.
- Los flujos de caracteres realizan una **conversión** entre bytes y caracteres.

Esta conversión depende de la codificación utilizada y es una fuente frecuente de errores si no se gestiona correctamente.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 5. CONCEPTO DE FLUJOS DE DATOS.

### 5.6 Encadenamiento de flujos

En muchos casos, los flujos se utilizan de forma **encadenada**, donde un flujo se apoya en otro para ampliar su funcionalidad.

Ejemplo conceptual:

- Un flujo básico accede al archivo.
- Otro flujo añade conversión o buffering.

Este encadenamiento permite:

- Mejorar el rendimiento.
- Facilitar el procesamiento de los datos.
- Adaptar el flujo a las necesidades del programa.

### 5.7 Cuándo es necesario usar flujos explícitos

El uso explícito de flujos es recomendable cuando:

- Los archivos son grandes.
- Se necesita leer o escribir progresivamente.
- Se requiere control detallado del proceso.
- Se procesan datos binarios complejos.

En estos casos, las operaciones de alto nivel no son suficientes.

### 5.8 Relación con las APIs de Java

Java proporciona distintas APIs basadas en flujos:

- APIs modernas que los usan internamente.
- APIs clásicas que los exponen directamente al programador.

Comprender el concepto de flujo permite:

- Entender la estructura de las APIs.
- Elegir la herramienta adecuada.
- Leer y escribir datos de forma correcta y eficiente.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 6. API CLÁSICA DE ENTRADA Y SALIDA (java.io).

### 6.1 Papel de la API `java.io`

La API **`java.io`** proporciona las clases fundamentales para trabajar **directamente con flujos de datos** en Java.

A diferencia de la API moderna:

- Aquí los flujos **son explícitos**.
- El programador controla:
  - Apertura
  - Lectura/escritura
  - Cierre del recurso

Esta API permite un control más detallado del proceso de entrada y salida.

### 6.2 Jerarquía básica de flujos

La API se organiza en dos grandes jerarquías:

**Flujos de bytes**

- `InputStream`
- `OutputStream`

**Flujos de caracteres**

- `Reader`
- `Writer`

Estas clases abstractas definen el comportamiento general de los flujos.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 6. API CLÁSICA DE ENTRADA Y SALIDA (java.io).

### 6.3 Flujos de bytes

Los flujos de bytes trabajan con **datos binarios**, sin interpretación como texto.

**Escritura de bytes: `FileOutputStream`**

Ejemplo:

```
try (FileOutputStream fos = new FileOutputStream("datos.bin")) {
    fos.write(65);   // escribe un byte
    fos.write(66);
} catch (IOException e) {
    e.printStackTrace();
}
```

En este caso:

- Se escriben valores numéricos directamente.
- El contenido no es legible como texto.

**Lectura de bytes: `FileInputStream`**

```
try (FileInputStream fis = new FileInputStream("datos.bin")) {
    int dato;
    while ((dato = fis.read()) != -1) {
        System.out.println(dato);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

La lectura se realiza byte a byte hasta llegar al final del archivo.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 6. API CLÁSICA DE ENTRADA Y SALIDA (java.io).

### 6.4 Flujos de caracteres

Los flujos de caracteres trabajan con **texto**, realizando internamente la conversión entre bytes y caracteres.

**Escritura de texto: `FileWriter`**

```
try (FileWriter writer = new FileWriter("texto.txt")) {
    writer.write("Primera línea\n");
    writer.write("Segunda línea\n");
} catch (IOException e) {
    e.printStackTrace();
}
```

Este enfoque es adecuado para:

- Escritura de texto sencillo.
- Archivos legibles.

**Lectura de texto: `FileReader`**

```
try (FileReader reader = new FileReader("texto.txt")) {
    int c;
    while ((c = reader.read()) != -1) {
        System.out.print((char) c);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

Aquí se lee carácter a carácter, lo que puede resultar poco eficiente.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 6. API CLÁSICA DE ENTRADA Y SALIDA (java.io).

### 6.5 Flujos con buffer

Para mejorar el rendimiento, se utilizan **flujos con buffer**, que reducen el número de accesos al archivo.

**Escritura con `BufferedWriter`**

```
try (BufferedWriter bw = new BufferedWriter(new FileWriter("texto.txt"))) {
    bw.write("Línea uno");
    bw.newLine();
    bw.write("Línea dos");
} catch (IOException e) {
    e.printStackTrace();
}
```

**Lectura con `BufferedReader`**

```
try (BufferedReader br = new BufferedReader(new FileReader("texto.txt"))) {
    String linea;
    while ((linea = br.readLine()) != null) {
        System.out.println(linea);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

Este enfoque permite:

- Leer línea a línea.
- Simplificar el procesamiento de texto.

#### 6.5.1 Escritura en modo append con FileWriter

```
new FileWriter("archivo.txt", true);
```

- `true` → concatena
- `false` / omitido → sobrescribe

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 6. API CLÁSICA DE ENTRADA Y SALIDA (java.io).

### 6.6 InputStreamReader: conversión de bytes a caracteres

`InputStreamReader` es una **clase puente** entre:

- los **flujos de bytes** (`InputStream`)
- y los **flujos de caracteres** (`Reader`)

**Adapta un flujo de bytes para que pueda leerse como texto**.
No accede directamente al archivo.
Se apoya siempre en un `InputStream` existente.
Relación jerárquica importante:

- `InputStreamReader` **hereda de `Reader`**
- Internamente **envuelve** un `InputStream`

**¿Qué problema real resuelve?**
En Java (y en cualquier sistema):

- **Los archivos almacenan bytes**
- **El texto trabaja con caracteres**

Esto genera un problema fundamental:
Un mismo byte puede representar cosas distintas según la codificación:
•El byte `0xE1` puede ser:

- `á` en ISO-8859-1
- parte de un carácter multibyte en UTF-8

**Si no se especifica la codificación**, el texto puede:

- leerse mal
- mostrar caracteres corruptos
- funcionar en un sistema y fallar en otro

**¿Qué hace exactamente `InputStreamReader`?**
`InputStreamReader` realiza **dos tareas críticas**:

1. **Lee bytes** desde un `InputStream`
2. **Los convierte en caracteres** usando una **codificación concreta**

Es decir:
`Bytes (archivo)` → `InputStreamReader` → `Caracteres (programa)`
La conversión depende **exclusivamente** del `Charset` indicado.

Constructor fundamental:

```
InputStreamReader(InputStream in, Charset cs)
```

Parámetros:

- `in` → flujo de bytes (archivo, red, etc.)
- `cs` → codificación de caracteres (UTF-8, UTF-16, ...)

```
InputStreamReader isr =
    new InputStreamReader(
        new FileInputStream("archivo.txt"),
        StandardCharsets.UTF_8
    );
```

Este código garantiza que:

- el archivo se interpreta como **UTF-8**
- el comportamiento es **idéntico en cualquier SO.**

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 6. API CLÁSICA DE ENTRADA Y SALIDA (java.io).

### 6.6 InputStreamReader: conversión de bytes a caracteres

**Comparación clave con `FileReader`**

`FileReader`:

- Es un **atajo**
- Usa la **codificación por defecto del sistema**
- No permite control explícito

`InputStreamReader`:

- Obliga a pensar en la codificación
- Hace el código **portable y profesional**
- Es la opción recomendada en aplicaciones reales

Si la codificación importa, **no uses `FileReader`**.

Lectura carácter a carácter (solo para entender el mecanismo):

```
try (InputStreamReader isr =
        new InputStreamReader(
            new FileInputStream("archivo.txt"),
            StandardCharsets.UTF_8
        )) {
    int c;
    while ((c = isr.read()) != -1) {
        System.out.print((char) c);
    }

} catch (IOException e) {
    e.printStackTrace();
}
```

- Se leen **caracteres**, no bytes
- Es correcto pero **poco eficiente**
- Se mejorará con `BufferedReader`

`InputStreamReader` es la **pieza central** que conecta el mundo binario del sistema de archivos con el mundo textual del programa.

Sin él:

- no hay control de codificación
- no hay portabilidad real
- no hay lectura profesional de texto

`FileInputStream↓InputStreamReader↓BufferedReader`

```
try (BufferedReader br =
        new BufferedReader(
            new InputStreamReader(
                new FileInputStream("archivo.txt"),
                StandardCharsets.UTF_8
            ))) {

    String linea;
    while ((linea = br.readLine()) != null) {
        System.out.println(linea);
    }

} catch (IOException e) {
    e.printStackTrace();
}
```

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 6. API CLÁSICA DE ENTRADA Y SALIDA (java.io).

### 6.7 OutputStreamWriter: conversión de caracteres a bytes

`OutputStreamWriter` es una **clase puente** entre:

- los **flujos de caracteres** (`Writer`)
- y los **flujos de bytes** (`OutputStream`)

Su función es **adaptar texto (caracteres) para que pueda escribirse como bytes** en un destino físico (archivo, red, etc.).

- No escribe directamente en el archivo.
- Siempre se apoya en un `OutputStream`.

Relación jerárquica:

- `OutputStreamWriter` **hereda de `Writer`**
- Internamente **envuelve** un `OutputStream`

**Qué problema real resuelve?**

Los programas trabajan con **caracteres**, pero:

- Los archivos **solo almacenan bytes**
- Cada carácter debe convertirse en uno o varios bytes

El problema aparece cuando **no se controla la codificación**:

- Un mismo carácter (`ñ`, `€`, `á`) puede ocupar:
  - 1 byte
  - varios bytes
- Dependiendo de la codificación usada

Si no se especifica la codificación:

- El archivo puede escribirse correctamente en un sistema
- Y leerse incorrectamente en otro.

`OutputStreamWriter` realiza **la conversión inversa** a `InputStreamReader`:

`Caracteres (programa)` → `OutputStreamWriter` → `Bytes (archivo)`

Sus responsabilidades son:

1. Recibir **caracteres**
2. Convertirlos a **bytes**
3. Escribirlos en el `OutputStream` subyacente
4. Usar **una codificación concreta**

Constructor fundamental:

```
OutputStreamWriter(OutputStream out, Charset cs)
```

Parámetros:

- `out` → flujo de bytes de salida
- `cs` → codificación de caracteres

```
OutputStreamWriter osw =
    new OutputStreamWriter(
        new FileOutputStream("archivo.txt"),
        StandardCharsets.UTF_8
    );
```

Este código garantiza que:

- el texto se escriba en **UTF-8**
- el archivo sea legible en cualquier sistema

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 6. API CLÁSICA DE ENTRADA Y SALIDA (java.io).

### 6.7 OutputStreamWriter: conversión de caracteres a bytes

**Comparación clave con `FileWriter`**

`FileWriter`:

- Es un **atajo**
- Usa la codificación por defecto del sistema
- Código más corto, pero menos control

`OutputStreamWriter`:

- Permite codificación explícita
- Es más verboso
- Es la opción correcta en código profesional

Si importa la codificación, **no uses `FileWriter`**.

```
try (OutputStreamWriter osw =
        new OutputStreamWriter(
            new FileOutputStream("archivo.txt"),
            StandardCharsets.UTF_8
        )) {

    osw.write("Primera línea\n");
    osw.write("Segunda línea\n");

} catch (IOException e) {
    e.printStackTrace();
```

- Se escriben **caracteres**
- Se convierten internamente en bytes
- Es correcto pero **no óptimo** para grandes volúmenes

**Relación con `BufferedWriter`**

En la práctica, `OutputStreamWriter` se utiliza así:

`FileOutputStream → OutputStreamWriter → BufferedWriter`

```
try (BufferedWriter bw =
        new BufferedWriter(
            new OutputStreamWriter(
                new FileOutputStream("archivo.txt"),
                StandardCharsets.UTF_8
            ))) {

    bw.write("Primera línea");
    bw.newLine();
    bw.write("Segunda línea");
    bw.newLine();

} catch (IOException e) {
    e.printStackTrace();
}
```

`OutputStreamWriter` garantiza que el texto que escribe el programa se convierta correctamente en bytes.

Es la **pieza imprescindible** para una escritura de texto:

- portable
- controlada
- profesional

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 6. API CLÁSICA DE ENTRADA Y SALIDA (java.io).

```java
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.nio.charset.StandardCharsets;

public class LecturaEscrituraTexto {
    private static final String FICHERO = "archivo.txt";
    public static void main(String[] args) {
        escribirArchivo();
        leerArchivo();
    }
    // Escritura de texto con patrón completo
    private static void escribirArchivo() {
        try (BufferedWriter bw =
                new BufferedWriter(
                    new OutputStreamWriter(
                        new FileOutputStream(FICHERO),
                        StandardCharsets.UTF_8
                    ))) {
            bw.write("Primera línea del archivo");
            bw.newLine();
            bw.write("Segunda línea con caracteres especiales: á é í ñ €");
            bw.newLine();
            bw.write("Tercera línea de prueba");

        } catch (IOException e) {
            System.err.println("Error al escribir el archivo");
            e.printStackTrace();
        }
    }
```

```java
    // Lectura de texto con patrón completo
    private static void leerArchivo() {
        try (BufferedReader br =
                new BufferedReader(
                    new InputStreamReader(
                        new FileInputStream(FICHERO),
                        StandardCharsets.UTF_8
                    ))) {
            String linea;
            System.out.println("Contenido del archivo:");
            while ((linea = br.readLine()) != null) {
                System.out.println(linea);
            }
        } catch (IOException e) {
            System.err.println("Error al leer el archivo");
            e.printStackTrace();
        }
    }
}
```

**Escritura**

- El programa **trabaja con caracteres**
- `OutputStreamWriter` los convierte en **bytes UTF-8**
- `BufferedWriter` mejora el rendimiento
- El archivo generado es **portable**

**Lectura**

- `FileInputStream` lee bytes del archivo
- `InputStreamReader` los convierte a caracteres **con la misma codificación**
- `BufferedReader` permite lectura por líneas

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 6. API CLÁSICA DE ENTRADA Y SALIDA (java.io).

### 6.8 Encadenamiento de flujos

En los ejemplos anteriores se observa el **encadenamiento de flujos**:

- Un flujo accede al archivo.
- Otro añade funcionalidad (buffer, conversión).

Ejemplo conceptual:

- `FileInputStream` → acceso al archivo
- `InputStreamReader` → conversión bytes → caracteres
- `BufferedReader` → lectura eficiente

### 6.9 Gestión de recursos

Los flujos deben cerrarse correctamente para:

- Liberar recursos.
- Evitar pérdidas de memoria.
- Garantizar la escritura completa de los datos.

Java proporciona `try-with-resources`, que:

- Cierra automáticamente los flujos.
- Simplifica el código.
- Reduce errores.

### 6.10 Limitaciones de la API clásica

Aunque sigue siendo válida, la API `java.io` presenta algunas limitaciones:

- Mayor cantidad de código.
- Gestión manual de flujos.
- Menor simplicidad para tareas comunes.

Por este motivo, suele reservarse para:

- Procesamiento progresivo.
- Archivos grandes.
- Casos donde se necesita control fino.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 7. ACCESO A DATOS BINARIOS ESTRUCTURADOS (*DataInputStream y DataOutputStream*).

### 7.1 Datos binarios estructurados

Un fichero binario puede almacenar datos como una simple secuencia de bytes, pero en muchos casos es necesario **darles una estructura lógica**.

Los **datos binarios estructurados** son aquellos en los que:

- Cada dato tiene un tipo concreto.
- El orden de escritura es conocido.
- La lectura debe realizarse en el mismo orden.

Este enfoque permite almacenar información de forma compacta y eficiente.

### 7.2 Papel de `DataOutputStream` y `DataInputStream`

Java proporciona dos clases para trabajar con datos binarios tipados:

- **`DataOutputStream`**: permite escribir datos primitivos en formato binario.
- **`DataInputStream`**: permite leer esos datos posteriormente.

Estas clases se apoyan en flujos de bytes y añaden métodos específicos para cada tipo de dato.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 7. ACCESO A DATOS BINARIOS ESTRUCTURADOS (*DataInputStream y DataOutputStream*).

### 7.3 Escritura de datos binarios con `DataOutputStream`

Ejemplo de escritura:

```
try (DataOutputStream dos = new DataOutputStream(
        new FileOutputStream("datos.bin"))) {

    dos.writeInt(25);
    dos.writeDouble(1850.75);
    dos.writeUTF("Juan");

} catch (IOException e) {
    e.printStackTrace();
}
```

Qué ocurre en este código:

- Se escriben un entero, un número decimal y una cadena.
- Cada dato ocupa un número fijo o conocido de bytes.
- El archivo resultante no es legible como texto.

### 7.4 Lectura de datos binarios con `DataInputStream`

Para leer el archivo, es imprescindible **respetar el mismo orden**:

```
try (DataInputStream dis = new DataInputStream(
        new FileInputStream("datos.bin"))) {

    int edad = dis.readInt();
    double salario = dis.readDouble();
    String nombre = dis.readUTF();

    System.out.println(edad);
    System.out.println(salario);
    System.out.println(nombre);

} catch (IOException e) {
    e.printStackTrace();
}
```

Si el orden no coincide:

- Se producirán errores.
- Los datos se interpretarán incorrectamente.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 7. ACCESO A DATOS BINARIOS ESTRUCTURADOS (*DataInputStream y DataOutputStream*).

### 7.5 Importancia del orden de lectura y escritura

En los ficheros binarios estructurados:

- **No hay separadores ni marcas visuales**.
- El programa debe conocer exactamente:
  - Qué tipo de dato se escribió.
  - En qué orden.

Un error en el orden provoca:

- Datos corruptos.
- Excepciones de lectura.
- Resultados incoherentes.

### 7.6 Uso de cadenas en binario

El método `writeUTF`:

- Escribe una cadena precedida de su longitud.
- Permite recuperarla correctamente con `readUTF`.

Esto facilita:

- Almacenar texto dentro de ficheros binarios.
- Combinar datos numéricos y textuales.

### 7.7 Casos de uso habituales

Este tipo de acceso es adecuado cuando:

- Se necesita eficiencia y tamaño reducido.
- Se controla completamente el formato del archivo.
- Los datos tienen una estructura fija.

Ejemplos:

- Archivos de configuración binarios simples.
- Formatos internos de aplicaciones.
- Datos compactos para uso exclusivo del programa.

### 7.8 Limitaciones del acceso binario estructurado

Aunque eficiente, presenta inconvenientes:

- Falta de legibilidad.
- Dependencia estricta del formato.
- Dificultad de mantenimiento.
- Escasa interoperabilidad.

Por estos motivos, se utiliza cuando:

- El rendimiento es prioritario.
- El archivo no está destinado a ser compartido.

# 8. ACCESO DIRECTO A ARCHIVOS (*RandomAccessFile*)

## 8.1 Acceso secuencial vs acceso directo

Hasta ahora, el acceso a archivos se ha realizado de forma **secuencial**:

- Los datos se leen o escriben desde el inicio hasta el final.
- Para llegar a un dato concreto es necesario recorrer los anteriores.

El **acceso directo** permite:

- Leer o escribir datos en **cualquier posición del archivo**.
- Evitar recorridos completos.
- Modificar información sin reescribir todo el fichero.

## 8.2 Qué es `RandomAccessFile`

`RandomAccessFile` es una clase que permite:

- Leer y escribir datos en un archivo.
- Acceder a posiciones concretas mediante un **puntero interno**.
- Trabajar con datos binarios tipados.

A diferencia de otras clases:

- No hereda de `InputStream` ni `OutputStream`.
- Combina lectura y escritura en una única clase.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 8. ACCESO DIRECTO A ARCHIVOS (*RandomAccessFile*)

### 8.3 Modos de apertura del archivo

Al crear un `RandomAccessFile`, se especifica el modo de acceso:

- `"r"` → solo lectura.
- `"rw"` → lectura y escritura.

Ejemplo:

```
RandomAccessFile raf = new
RandomAccessFile("datos.dat", "rw");
```

### 8.4 El puntero del archivo

`RandomAccessFile` mantiene un **puntero interno** que indica la posición actual dentro del archivo.

Métodos principales:

- `getFilePointer()` → devuelve la posición actual.
- `seek(long pos)` → mueve el puntero a una posición concreta.

Este puntero determina dónde se leerá o escribirá el siguiente dato.

### 8.5 Escritura en posiciones concretas

Ejemplo de escritura:

```
try (RandomAccessFile raf = new
RandomAccessFile("datos.dat", "rw")) {

    raf.seek(0);
    raf.writeInt(100);

    raf.seek(4);
    raf.writeDouble(75.5);

    raf.seek(12);
    raf.writeUTF("Registro1");

} catch (IOException e) {
    e.printStackTrace();
}
```

Explicación:

- `int` ocupa 4 bytes.
- `double` ocupa 8 bytes.
- La posición 12 corresponde al final del `double`.

La correcta gestión de posiciones es esencial.

# 8. ACCESO DIRECTO A ARCHIVOS (*RandomAccessFile*)

## 8.6 Lectura desde posiciones concretas

Para leer los datos, se debe respetar:

- El tipo de dato.
- La posición en la que se escribió.

```
try (RandomAccessFile raf = new RandomAccessFile("datos.dat", "r")) {
    raf.seek(0);
    int numero = raf.readInt();
    raf.seek(4);
    double valor = raf.readDouble();
    raf.seek(12);
    String texto = raf.readUTF();
    System.out.println(numero);
    System.out.println(valor);
    System.out.println(texto);
} catch (IOException e) {
    e.printStackTrace(); }
```

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 8. ACCESO DIRECTO A ARCHIVOS (*RandomAccessFile*)

```
import java.io.IOException;
import java.io.RandomAccessFile;

public class RandomAccessFileExample {
    public static void main(String[] args) {
        String filePath = "archivo.dat";

        // Escribir datos en posiciones específicas del archivo
        escribirEnArchivo(filePath);

        // Leer datos desde posiciones específicas del archivo
        leerDesdeArchivo(filePath);
    }

public static void escribirEnArchivo(String filePath) {
        try (RandomAccessFile file = new RandomAccessFile(filePath, "rw")) {
            // Escribir en diferentes posiciones
            file.seek(0);   // Moverse al inicio del archivo
            file.writeInt(100); // Escribir un entero en la posición 0

            file.seek(4);   // Moverse a la siguiente posición después del entero (4 bytes)
            file.writeDouble(55.78); // Escribir un double en la posición 4

            file.seek(12);  // Moverse a la siguiente posición después del double (8 bytes)
            file.writeUTF("Hola Mundo"); // Escribir una cadena de texto en la posición 12

            System.out.println("Datos escritos correctamente en el archivo.");
        } catch (IOException e) {
            System.err.println("Error al escribir en el archivo: " + e.getMessage());
        }
    }
}
```

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 8. ACCESO DIRECTO A ARCHIVOS (*RandomAccessFile*)

```
public static void leerDesdeArchivo(String filePath) {
        try (RandomAccessFile file = new RandomAccessFile(filePath, "r")) {
            // Leer en el mismo orden en que fueron escritos
            file.seek(0);
            int numero = file.readInt();  // Leer el entero de la posición 0

            file.seek(4);
            double decimal = file.readDouble();  // Leer el double de la posición 4

            file.seek(12);
            String texto = file.readUTF();  // Leer la cadena de texto de la posición 12

            // Mostrar los datos leídos
            System.out.println("Número entero leído: " + numero);
            System.out.println("Número decimal leído: " + decimal);
            System.out.println("Texto leído: " + texto);
        } catch (IOException e) {
            System.err.println("Error al leer el archivo: " + e.getMessage());
        }
    }
}
```

**Explicación del código:**

- Abrimos el archivo con RandomAccessFile en modo lectura/escritura ("rw").
- Usamos seek(long pos) para movernos a una posición específica en el archivo.
- Escribimos diferentes tipos de datos (int, double, String) en posiciones específicas.
- Luego, leemos los datos desde esas mismas posiciones para demostrar que podemos acceder a ellos sin recorrer todo el archivo.
- Los datos se imprimen en consola para verificar que la escritura y lectura fueron exitosas.

# 8. ACCESO DIRECTO A ARCHIVOS (*RandomAccessFile*)

## 8.7 Uso de registros de tamaño fijo

Una aplicación habitual de `RandomAccessFile` es el uso de **registros de tamaño fijo**.

En este enfoque:

- Cada registro ocupa el mismo número de bytes.
- La posición de un registro se calcula fácilmente.

Ejemplo conceptual:

```
posición = númeroRegistro × tamañoRegistro
```

Esto permite:

- Acceso rápido a cualquier registro.
- Modificación directa de datos.

## 8.8 Casos de uso habituales

El acceso directo es adecuado cuando:

- Se manejan archivos grandes.
- Se requiere modificar partes concretas.
- Se implementan estructuras simples de almacenamiento.

Ejemplos:

- Ficheros de índices.
- Almacenamiento binario estructurado.
- Simulación de bases de datos simples.

## 8.9 Limitaciones de `RandomAccessFile`

A pesar de su potencia, presenta limitaciones:

- Gestión manual del formato.
- Dependencia del tamaño exacto de los datos.
- Dificultad para manejar estructuras variables.
- No escala bien para aplicaciones complejas.

Por este motivo, su uso se reserva a casos muy concretos.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 9. PERSISTENCIA DE OBJETOS (SERIALIZACIÓN)

### 9.1 Persistencia de objetos: concepto y contexto

Hasta ahora hemos trabajado con:

- datos sueltos (texto, números)
- estructuras planas (líneas, registros)

Sin embargo, en programación orientada a objetos **la unidad natural de información es el objeto**.

**Problema real**

Un objeto en memoria:

- existe solo durante la ejecución
- desaparece al cerrar el programa

Ejemplo:

```
Persona p = new Persona("Ana", 30);
```

Cuando el programa termina:

- el objeto se pierde
- no puede recuperarse

**Solución: serialización**

La **serialización** permite:

Guardar un objeto completo (atributos y estado) convirtiéndolo en una secuencia de bytes.

# 9. PERSISTENCIA DE OBJETOS (SERIALIZACIÓN)

## 9.2 Interfaz `Serializable` y requisitos

Para que una clase pueda serializarse:

```
import java.io.Serializable;

public class Persona implements Serializable {
    private String nombre;
    private int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    @Override
    public String toString() {
        return nombre + " (" + edad + ")";
    }
}
```

Aspectos importantes:

- `Serializable` **no tiene métodos**
- Actúa como **marca**
- Todos los atributos también deben ser serializables

Si un atributo no lo es → `NotSerializableException`

# 9. PERSISTENCIA DE OBJETOS (SERIALIZACIÓN)

## 9.3 Escritura de objetos en archivo (serialización)

```
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectOutputStream;

public class GuardarPersona {

    public static void main(String[] args) {
        Persona p = new Persona("Ana", 30);

        try (ObjectOutputStream oos =
                 new ObjectOutputStream(
                     new FileOutputStream("persona.dat"))) {

            oos.writeObject(p);

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Qué ocurre internamente:

1. El objeto se inspecciona
2. Se convierte a bytes
3. Se guarda su estado completo
4. El archivo resultante es **binario**

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 9. PERSISTENCIA DE OBJETOS (SERIALIZACIÓN)

### 9.4 Lectura de objetos (deserialización)

```
import java.io.FileInputStream;
import java.io.IOException;
import java.io.ObjectInputStream;

public class LeerPersona {

    public static void main(String[] args) {

        try (ObjectInputStream ois =
                 new ObjectInputStream(
                     new FileInputStream("persona.dat")))
{

            Persona p = (Persona) ois.readObject();
            System.out.println(p);

        } catch (IOException | ClassNotFoundException e)
{
            e.printStackTrace();
        }
    }
}
```

Reglas críticas:

- La clase debe existir
- Debe ser compatible con la versión usada al serializar
- Cambios en la clase pueden romper la lectura

### 9.5 Limitaciones reales de la serialización

Problemas importantes:

- Dependencia total de Java
- Archivos ilegibles
- Versionado complejo
- No interoperable

La serialización **no es un formato de intercambio**, es una solución técnica puntual.

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 9. PERSISTENCIA DE OBJETOS (SERIALIZACIÓN)

### 9.6. PERSISTENCIA MODERNA CON JSON

JSON representa objetos mediante **texto estructurado**:

```
{
  "nombre": "Ana",
  "edad": 30
}
```

Ventajas reales:

- Legible por personas
- Independiente del lenguaje
- Fácil de depurar
- Estándar de facto en la industria

**De objeto Java a JSON (concepto práctico)**

Supongamos esta clase:

```
public class Persona {
    private String nombre;
    private int edad;
}
```

Representación JSON equivalente:

```
{
  "nombre": "Ana",
  "edad": 30
}
```

Proceso lógico:

`Objeto → JSON → Archivo Archivo → JSON → Objeto`

**Ejemplo práctico: guardar objeto como JSON (sin librerías externas)**

```
try (BufferedWriter bw =
        new BufferedWriter(
            new OutputStreamWriter(
                new FileOutputStream("persona.json"),
                StandardCharsets.UTF_8))) {

    bw.write("{");
    bw.newLine();
    bw.write("  \"nombre\": \"Ana\",");
    bw.newLine();
    bw.write("  \"edad\": 30");
    bw.newLine();
    bw.write("}");

} catch (IOException e) {
    e.printStackTrace();
}
```

Este ejemplo:

- conecta con lo visto en flujos
- muestra claramente el formato
- prepara el terreno para usar librerías reales

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 9. PERSISTENCIA DE OBJETOS (SERIALIZACIÓN)

### 9.6. PERSISTENCIA MODERNA CON JSON

#### SERIALIZACIÓN CON GSON

Gson es una librería:

- ligera
- muy fácil de usar
- ideal para aprendizaje y proyectos medios

Convierte automáticamente:

`Objeto Java ↔ JSON`

```
public class Persona {
    private String nombre;
    private int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    public String getNombre() { return nombre; }
    public int getEdad() { return edad; }
}
```

**Objeto → JSON (serialización)**

```
import com.google.gson.Gson;
import java.io.FileWriter;
import java.io.IOException;

public class GuardarConGson {

    public static void main(String[] args) {
        Persona p = new Persona("Ana", 30);
        Gson gson = new Gson();

        try (FileWriter writer = new
FileWriter("persona.json")) {
            gson.toJson(p, writer);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Resultado (`persona.json`):

`{"nombre":"Ana","edad":30}`

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 9. PERSISTENCIA DE OBJETOS (SERIALIZACIÓN)

### 9.6. PERSISTENCIA MODERNA CON JSON

#### SERIALIZACIÓN CON GSON

**JSON → Objeto (deserialización)**

```
import com.google.gson.Gson;
import java.io.FileReader;
import java.io.IOException;

public class LeerConGson {

    public static void main(String[] args) {
        Gson gson = new Gson();

        try (FileReader reader = new
FileReader("persona.json")) {
            Persona p = gson.fromJson(reader,
Persona.class);
            System.out.println(p.getNombre());
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

**Cuándo usar Gson**

- Proyectos educativos
- Aplicaciones pequeñas y medianas
- Código sencillo
- Curva de aprendizaje baja

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 9. PERSISTENCIA DE OBJETOS (SERIALIZACIÓN)

### 9.6. PERSISTENCIA MODERNA CON JSON

#### SERIALIZACIÓN CON JACKSON

Jackson es una librería:

- más potente
- más configurable
- estándar de facto en **Spring Boot**
- usada en backend profesional

Clase de ejemplo (misma que antes)

Jackson trabaja con:

- getters/setters
- o campos directamente

**Objeto → JSON con Jackson**

```
import com.fasterxml.jackson.databind.ObjectMapper;
import java.io.File;

public class GuardarConJackson {

    public static void main(String[] args) {
        Persona p = new Persona("Ana", 30);
        ObjectMapper mapper = new ObjectMapper();

        try {
            mapper.writeValue(new File("persona.json"), p);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

**Resultado** (`persona.json`):

```
{
  "nombre" : "Ana",
  "edad" : 30
}
```

# LECTURA Y ESCRITURA DE INFORMACIÓN

## 9. PERSISTENCIA DE OBJETOS (SERIALIZACIÓN)

### 9.6. PERSISTENCIA MODERNA CON JSON

#### SERIALIZACIÓN CON JACKSON

**JSON → Objeto con Jackson**

```
import com.fasterxml.jackson.databind.ObjectMapper;
import java.io.File;

public class LeerConJackson {

    public static void main(String[] args) {
        ObjectMapper mapper = new ObjectMapper();

        try {
            Persona p = mapper.readValue(
                new File("persona.json"),
                Persona.class
            );
            System.out.println(p.getNombre());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

**4.5 Cuándo usar Jackson**

- Backend profesional
- APIs REST
- Spring / Spring Boot
- Necesidad de control avanzado

# LECTURA Y ESCRITURA DE INFORMACIÓN

## ALMACENAMIENTO DE OBJETOS EN FICHEROS. PERSISTENCIA. SERIALIZACIÓN

```java
public class TestFichero2 {
   public static class amigo implements java.io.Serializable{
      protected String nombre;
      protected long telefono;
      public amigo(String n, long t){
         nombre = n;
         telefono = t;
      }
       public void print(){
          System.out.println(nombre + " -> " + telefono);
       }
   }

    public static void main(String [] args) {
       String [] amigos = {"Andres Rosique", "Pedro Ruiz",
                           "Isaac Sanchez", "Juan Serrano"};
       long [] telefonos = {653364787, 627273746, 645424234, 612334452};
       try{
          FileOutputStream fs = new FileOutputStream("amigos.txt");
          ObjectOutputStream cos = new ObjectOutputStream(fs);
          for (int i = 0; i<4;i++){
             amigo a = new amigo(amigos[i],telefonos[i]);
             cos.writeObject(a);
          }
          if (cos!=null){
             cos.close();
             fs.close();
          }
       }catch(IOException e){e.printStackTrace();}
          try{
             File f = null;
             FileInputStream fe = null;
             ObjectInputStream ois = null;
             try{
                f = new File("amigos.txt");
                if(f.exists()){
                   fe = new FileInputStream(f);
                   ois = new ObjectInputStream(fe);
                   while(true){
                      amigo a = null;
                      a = (amigo) ois.readObject();
                      a.print();
                      System.out.println("");
                   }
                }
             }catch(EOFException eof){
                 System.out.println("---------");
             }catch(FileNotFoundException fnf){
                 System.err.println("Fichero no encontrado " + fnf);
             }catch(IOException e){
                 System.err.println("Se ha producido una excepción de e-s");
                 e.printStackTrace();
             }catch(Throwable e){
                 System.out.println("Error de programa: " + e);
                 e.printStackTrace();
             }finally{
                if (ois != null){
                   ois.close();
                   fe.close();
                }
             }
          }catch(IOException e){
             e.printStackTrace();
          }
-}}
```

```
run:
Andres Rosique -> 653364787

Pedro Ruiz -> 627273746

Isaac Sanchez -> 645424234

Juan Serrano -> 612334452

---------
BUILD SUCCESSFUL (total time: 0 seconds)
```

# Serialización con JSON (Jackson, Gson)

## 1 Serialización y deserialización con Jackson

```
import com.fasterxml.jackson.databind.ObjectMapper;
import java.io.IOException;

public class JacksonExample {
    public static void main(String[] args) {
        // Crear un objeto Ejemplo
        Persona persona = new Persona("Juan", 30, "juan@example.com");

        try {
            // Crear el ObjectMapper (necesario para Jackson)
            ObjectMapper mapper = new ObjectMapper();

            // Serializar el objeto a JSON
            String json = mapper.writeValueAsString(persona);
            System.out.println("JSON generado con Jackson: " + json);

            // Deserializar JSON a objeto
            Persona personaRecuperada = mapper.readValue(json, Persona.class);
            System.out.println("Objeto deserializado con Jackson: " + personaRecuperada);

        } catch (IOException e) {
            System.err.println("Error al procesar JSON: " + e.getMessage());
        }
    }
}
```

# LECTURA Y ESCRITURA DE INFORMACIÓN

## ALMACENAMIENTO DE OBJETOS EN FICHEROS. PERSISTENCIA. SERIALIZACIÓN

# Serialización con JSON (Jackson, Gson)

### 1 Serialización y deserialización con Jackson

```
// Clase de ejemplo para serialización
class Persona {
    private String nombre;
    private int edad;
    private String email;

    // Constructor vacío (Jackson lo requiere)
    public Persona() {}

    public Persona(String nombre, int edad, String email) {
        this.nombre = nombre;
        this.edad = edad;
        this.email = email;
    }

    // Getters y Setters (necesarios para la serialización/deserialización)
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public int getEdad() { return edad; }
    public void setEdad(int edad) { this.edad = edad; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    @Override
    public String toString() {
        return "Persona{nombre='" + nombre + "', edad=" + edad + ", email='" + email + "'}";
    }
}
```

# LECTURA Y ESCRITURA DE INFORMACIÓN

## ALMACENAMIENTO DE OBJETOS EN FICHEROS. PERSISTENCIA. SERIALIZACIÓN

## Serialización con JSON (Jackson, Gson)

### 2 Serialización y deserialización con Gson

```
import com.google.gson.Gson;

public class GsonExample {
    public static void main(String[] args) {
        // Crear un objeto Ejemplo
        Persona persona = new Persona("Ana", 25, "ana@example.com");

        // Crear instancia de Gson
        Gson gson = new Gson();

        // Serializar el objeto a JSON
        String json = gson.toJson(persona);
        System.out.println("JSON generado con Gson: " + json);

        // Deserializar JSON a objeto
        Persona personaRecuperada = gson.fromJson(json, Persona.class);
        System.out.println("Objeto deserializado con Gson: " + personaRecuperada);
    }
}
```

# LECTURA Y ESCRITURA DE INFORMACIÓN

## ALMACENAMIENTO DE OBJETOS EN FICHEROS. PERSISTENCIA. SERIALIZACIÓN

## Serialización con JSON (Jackson, Gson)

**Explicación del código**:

**Clase Persona**: Define un objeto con atributos nombre, edad y email.

Se incluyen constructores, getters y setters (Jackson los necesita).

El método toString() ayuda a imprimir el objeto de manera legible.

**Método con Jackson (ObjectMapper)**:

writeValueAsString(objeto) → Convierte un objeto en JSON.

readValue(json, Clase.class) → Convierte JSON en un objeto Java.

**Método con Gson (Gson)**:

toJson(objeto) → Serializa un objeto en JSON.

fromJson(json, Clase.class) → Deserializa JSON en un objeto Java.

# TEMA 6: LECTURA Y ESCRITURA DE INFORMACIÓN